

This program is a **minimal LangChain chat example without memory**. Its purpose is to show that an LLM can answer a single prompt well, but it does **not automatically remember earlier conversation turns** unless you explicitly provide that history.

What the program does overall

It:

1. Installs the required LangChain libraries.
2. Creates an OpenAI chat model wrapper.
3. Builds a simple prompt template.
4. Connects the prompt to the model in a chain.
5. Sends one message: “Hi, my name is Alice.”
6. Sends a second message: “What is my name?”
7. Demonstrates the problem: the second request may fail because the first message was not saved and reused.

Detailed explanation, step by step

1) Installing packages

```
!pip install -qU langchain langchain-openai langchain-core
```

This installs the core LangChain libraries needed for the notebook.

- langchain: the main framework
- langchain-openai: integration with OpenAI chat models
- langchain-core: prompt and runnable building blocks

Why it matters:

Without these packages, Colab does not know what ChatOpenAI or ChatPromptTemplate means.

2) Import statements

- import os
- from langchain_openai import ChatOpenAI
- from langchain_core.prompts import ChatPromptTemplate

What each one does:

- os: lets Python read environment variables, such as your API key.
- ChatOpenAI: LangChain’s wrapper for calling an OpenAI chat model.
- ChatPromptTemplate: a structured way to create prompts with variables.

Why it matters:

This keeps the code organized and avoids manually building raw prompt strings every time.

3) API key check

Python

```
if not os.getenv("OPENAI_API_KEY"):
    raise ValueError("Please set OPENAI_API_KEY first.")
```

This checks whether the environment variable OPENAI_API_KEY exists.

What it does:

- If the key is missing, the program stops immediately.
- If the key exists, execution continues.

Why it matters:

The model call needs authentication. Without the key, the program cannot contact the API.

Good practice:

This is better than failing later with a vague authentication error.

4) Creating the model

```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

This creates the language model object.

Key parts:

- model="gpt-4o-mini": selects which model to use.
- temperature=0: makes responses more deterministic and less random.

Why temperature=0 is useful here:

This example is educational. You want stable behavior so the memory issue is easier to observe.

5) Creating the prompt template

Python

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant. Keep answers short."),
    ("human", "{input}")
])
```

This defines the structure of every request sent to the model.

There are two message types here:

System message

"You are a helpful assistant. Keep answers short."

This sets behavior and tone.

It tells the model:

- act like a helpful assistant
- keep responses brief

Human message

"{input}"

This is a placeholder.

Whatever value you pass for input gets inserted here.

Why it matters:

The prompt template separates:

- fixed instructions
- dynamic user input

That makes the code reusable.

6) Building the chain

chain = prompt | llm

This is LangChain's pipe syntax.

What it means:

- First, format the prompt.
- Then, send the formatted prompt into the model.

Think of it like a pipeline:

input → prompt template → model → response

Why it matters:

This is the main LangChain idea: combine components into a flow.

What happens in Turn 1

7) First invocation

```
response1 = chain.invoke({"input": "Hi, my name is Alice."})
```

This sends one input into the chain.

What LangChain does internally:

1. Replaces {input} with "Hi, my name is Alice."
2. Builds the message list:
 - system: "You are a helpful assistant. Keep answers short."
 - human: "Hi, my name is Alice."
3. Sends those messages to the model
4. Receives a response object

Then:

```
print("AI:", response1.content)
```

This prints only the text content of the model's reply.

What the model will likely do:

It may respond with something like:

- "Nice to meet you, Alice."
- "Hello, Alice."

Why this works:

The model has everything it needs for that single turn.

What happens in Turn 2

8) Second invocation

```
response2 = chain.invoke({"input": "What is my name?"})
```

This looks like a follow-up question, but technically it is **not** part of the first conversation.

What LangChain sends this time:

- system: "You are a helpful assistant. Keep answers short."
- human: "What is my name?"

That is all.

What is missing:

The earlier message:

- "Hi, my name is Alice."

So the model may say:

- "I don't know your name."
- "You haven't told me your name."
- or guess incorrectly

The core issue: no memory

This is the main teaching point of the program.

Even though *you* see two turns in the notebook, the model sees them as **two separate requests** because the code does not pass prior conversation history into the second call.

Important concept

LLMs are typically **stateless by default**.

That means:

- They do not “remember” earlier calls automatically.
- Every call only knows what you include in that specific request.

So this program demonstrates a common beginner misunderstanding:

“I asked two questions in sequence, so the model should remember.”

Not true unless you add memory or manually pass previous messages.

Why the second call may fail

Because the prompt template only includes:

- one system message
- one current human input

It does **not** include:

- prior user messages
- prior assistant messages
- any stored session history

So the model has no basis for answering “What is my name?”

Why this program is still useful

This is actually a very good teaching example because it clearly shows:

- how to create a simple LangChain chain
- how prompt templates work
- how `.invoke()` is used
- why memory matters in chat applications

It is the simplest way to understand the difference between:

- **single-turn prompting**

- **multi-turn conversation**

What response.content means

The model returns a response object, not just plain text.

response1.content and response2.content extract the actual text reply.

Why it matters:

If you print the full object, you may see extra metadata. Using .content keeps the output clean.

Expected output behavior

You will likely see:

- Turn 1: a correct acknowledgment of “Alice”
- Turn 2: uncertainty or failure to recall the name

That mismatch is intentional. The program is designed to expose the memory gap.

Main limitation of this program

It cannot support true conversation.

It works for:

- isolated prompts
- simple demonstrations
- one-shot tasks

It does not work well for:

- chatbots
- assistants
- multi-turn Q&A
- personalized interactions

Simple mental model

Treat this program like calling the assistant on two separate phone calls where the assistant has no notes from the first call.

Call 1:

- “My name is Alice.”

Call 2:

- “What is my name?”

If no notes were saved, the assistant cannot know.

That is exactly what is happening here.

Bottom line

This program is a **stateless LangChain example**.

It shows that LangChain can easily connect a prompt template to a model, but **conversation memory is not automatic**. If you want the assistant to remember earlier turns, you need to add chat history or a memory mechanism explicitly.